ПРЕДЛОГ ЗА ДИПЛОМСКА РАБОТА

Event-driven микросервисна архитектура со модерна оркестрација и observability

Дизајн, имплементација и евалуација — студија на случај AutoMarket, платформа за огласи за автомобили

[Име Презиме] · [бр. на индекс]

Ментор: [проф. д-р Име Презиме]

[Факултет] · октомври 2026



Лесни за цртање, тешки за правилна изведба

Поделбата на сервиси на дијаграм е брза. Вистинската работа почнува кога нешто ќе падне.



Дистрибуиран монолит

Сервисите се одвоени на дијаграмот, но делат табели и синхрони повици — падот на еден ги урива и другите.



Изгубени настани

Запис во база и порака до брокер се две одделни операции. Ако втората не успее, системот тивко станува неконзистентен.



Невидливи дефекти

API-то враќа 200, а настаните престанале да течат. Без метрики, логови и traces, никој не забележува.

Темата на трудот: како се гради — и докажува — систем што останува коректен и кога нешто паѓа.

02 · ПОЧЕТНА ТОЧКА

Истата функционалност, изградена двапати

Стариот проект ми е основа за споредба — од него веќе знам што треба да прави системот.

ПОСТОИ ДЕНЕС · МОНОЛИТ

.NET · Razor Pages
серверски рендерирани страници

Една апликација
сета логика заедно

Една база
SQL

ПРЕДЛОГОТ · МИКРОСЕРВИСИ

Angular SPA
клиент

Apache Kafka
настани меѓу сервисите

API Gateway
JWT · CORS · rate limiting

Kubernetes
оркестрација · GitOps

6 сервиси
шема по сервис

Observability
метрики · логови · traces

Работен прототип на новата архитектура веќе постои, а дел од инфраструктурата беше претставен на предметот DevOps — темата е изводлива во рокот.

Цел и истражувачки прашања



ЦЕЛ НА ТРУДОТ

Да се дизајнира, изгради и евалуира production-grade микросервисна архитектура заснована на настани — оркестрирана со Kubernetes и целосно набљудлива — и да се спореди со монолитната верзија на истата апликација.

П1 Дали transactional outbox гарантира нула изгубени настани кога брокерот паѓа?

П2 Дали идемпотентните consumers даваат ефект „точно еднаш“ при at-least-once достава?

П3 Колкава е цената на дистрибуцијата — латенција, пропусност, ресурси — наспроти монолитот?

П4 Колку брзо observability стекот открива и лоцира дефект во асинхрониот тек?

04 · ДЕКОМПОЗИЦИЈА

Поделбата е по домен, не по слоеви

Секој сервис ги поседува своите табели, во своја PostgreSQL шема. Никој не чита туѓи табели.

API Gateway — единствена влезна точка

JWT валидација · CORS · rate limiting · рутирање

Корисници

`auth`

регистрација, најава, JWT, улоги

Огласи

`listing`

огласи, слики, омилени, модерација

Блог

`blog`

објави и уреднички содржини

Разговори

`inquiry`

пораки купувач ↔ продавач

Плаќања

`payment`

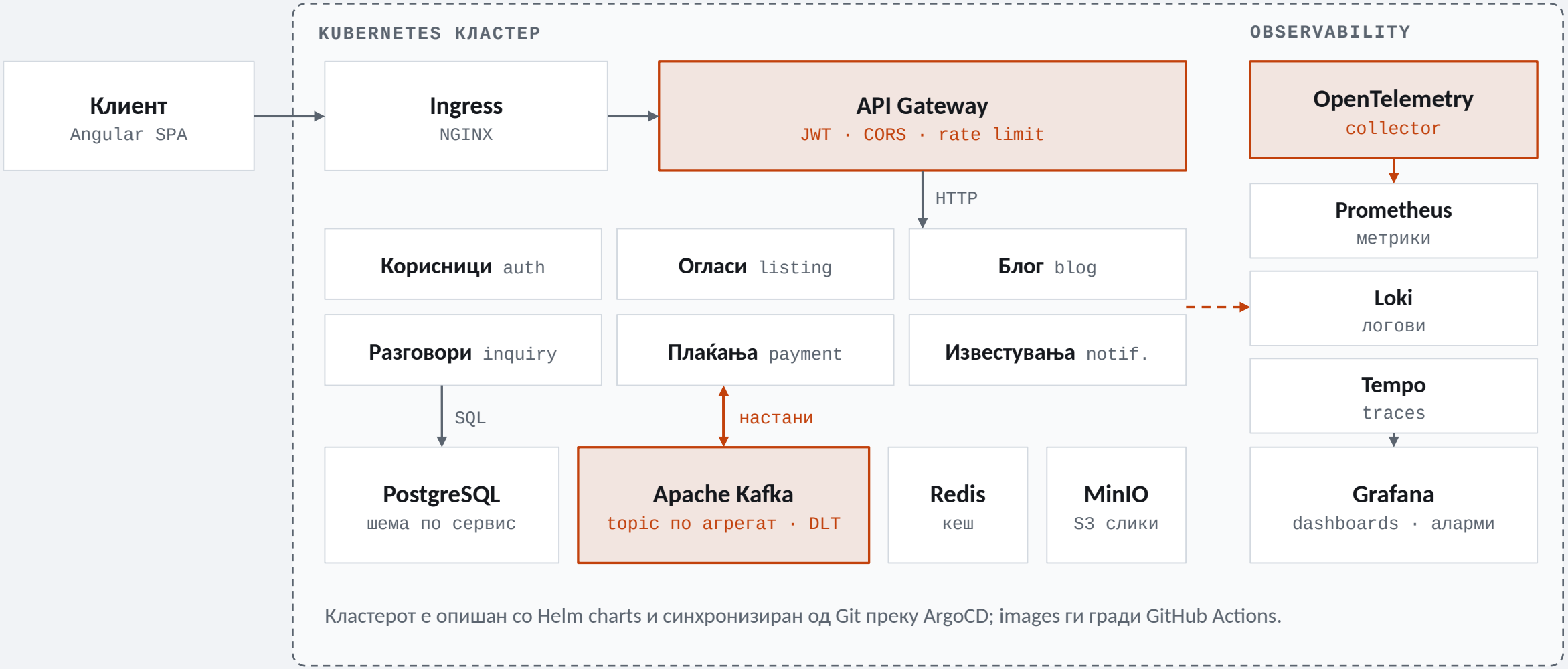
претплати преку Stripe

Известувања

`notification · без база`

email известувања од настани

Целна архитектура



Секој сервис чува копија од тоа што му треба

СЕРВИС: КОРИСНИЦИ

табела „корисници“

`id · име · телефон · email`

Огласот мора да ги прикаже името и телефонот на продавачот — а сервисот за огласи го има само продавач_id.

НАСТАН ПРИ РЕГИСТРАЦИЈА

„корисник 41: Марко, 070…“

преку Kafka, асинхроно

СЕРВИС: ОГЛАСИ

табела „огласи“

`id · марка · цена · продавач_id`

НОВА ТАБЕЛА · ЛОКАЛНА ПРОЕКЦИЈА

табела „продавачи“

`id · име · телефон`

Да прашува секој пат

синхрон REST повик

Ако сервисот за корисници не работи, не се отвораат ни огласите. Падот се шири низ системот.

Да запише однапред

настан + локална проекција

Огласот се отвора само од своите табели и не прашува никого. Цената: копијата кратко доцни (eventual consistency).

Што ако пораката не се прати?

ПРОБЛЕМОТ · DUAL WRITE

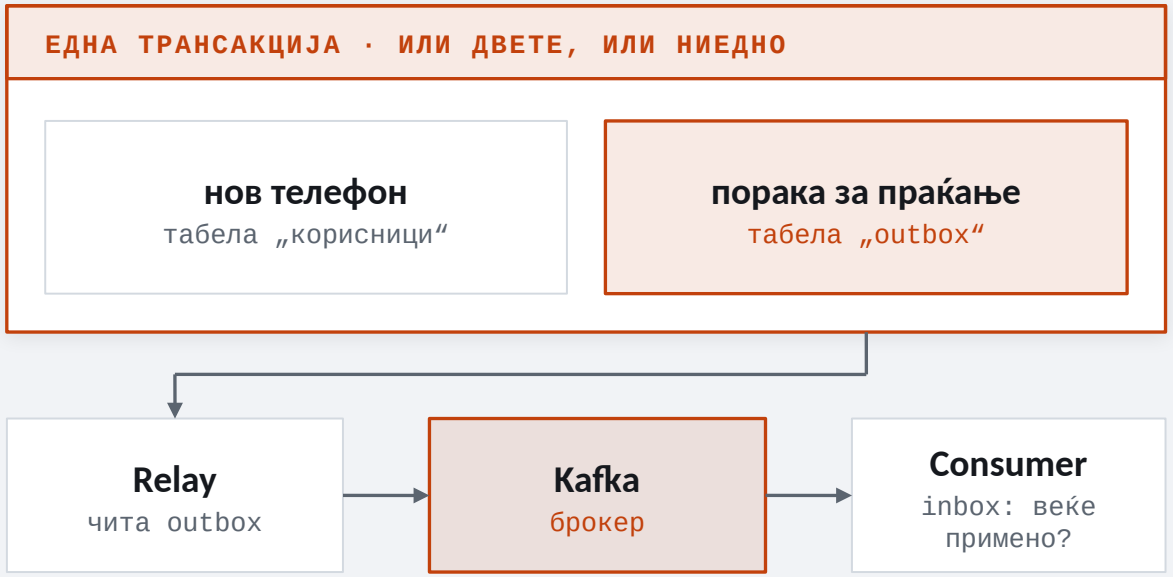
Марко си го менува телефонот. Две работи треба да се случат, една по друга:

1. Сервисот за корисници го запишува новиот телефон во својата табела.
   готово ✓

2. Истиот сервис праќа порака до Kafka, за да дознае и сервисот за огласи.
   тука може да падне ×

Чекор 1 не се враќа назад. Телефонот е нов во едниот сервис, а стар во другиот — и никаде не пишува дека тоа се случило.

РЕШЕНИЕТО · TRANSACTIONAL OUTBOX

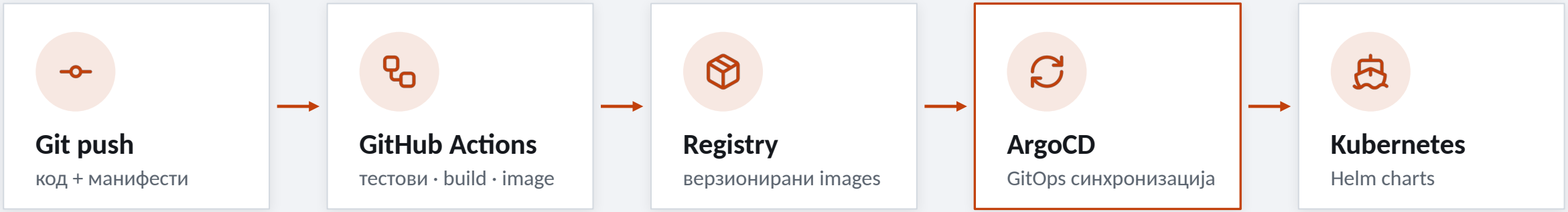


Kafka не работи → пораката чека во outbox и се праќа кога брокерот ќе се врати. Ништо не се губи — само доцни.

Иста порака двапати → inbox ја препознава по id; ефектот се случува само еднаш.

Од commit до кластер, без рачни чекори

Секоја промена поминува низ ист автоматизиран пат, а посакуваната состојба на кластерот е запишана во Git.



ВО КЛАСТЕРОТ

Health проби
сообраќај само до здрави pods

Автоскалирање (HPA)
реплики според оптоварувањето

PodDisruptionBudget
достапност и при одржување

NetworkPolicy
default-deny меѓу сервисите

ConfigMap и Secret
конфигурација надвор од кодот

Ingress
една влезна точка за клиентот

Три столба: метрики, логови, traces

Асинхрониот систем греши тивко. Целта е секој дефект да се забележи, објасни и лоцира.

PROMETHEUS

Метрики

„Дали нешто не е во ред?“

латенција и грешки по сервис, Kafka consumer lag, старост на outbox

LOKI

Логови

„Што точно се случило?“

структурирани JSON логови од сите pods, пребарливи по trace id

TEMPO

Traces

„Каде се случило?“

едно барање следено низ gateway → сервис → Kafka → consumers

OpenTelemetry

една инструментација и еден collector за сите три сигнали — независно од алатката што ги чува

Grafana: dashboards и аларми

аларми за дефекти што однадвор не се гледаат:

`OutboxStalled` · `ConsumerLag` · `DeadLetter`

Технолошки стек

Апликација

Java 21 · Spring Boot 3
Spring Cloud Gateway
Angular (SPA)

Податоци

PostgreSQL 16
шема по сервис · Flyway
Redis · MinIO (S3)

Настани

Apache Kafka (KRaft)
Outbox / Inbox
Dead-letter topics

Оркестрација

Docker
Kubernetes (k3d)
Helm

CI/CD и GitOps

GitHub Actions
Container registry
ArgoCD

Observability

OpenTelemetry
Prometheus · Grafana
Loki · Tempo

Тестови и евалуација

JUnit · Testcontainers
k6 — оптоварување
Chaos Mesh — откази

Интеграции

Stripe (test mode)
SMTP · MailHog
JWT (RS256)

Четири проверки со бројка на крајот

Не „еве, работи“ — секоја проверка одговара на едно истражувачко прашање и завршува со мерлив резултат.

ПРОВЕРКА 1 · П1

Гасам Kafka среде операција

Chaos Mesh

0

изгубени настани

ПРОВЕРКА 2 · П2

Ист настан праќам двапати

повторена достава

1

известување, не две

ПРОВЕРКА 3 · П3

Оптоварување врз двете верзии

k6

p95

латенција, пропусност и CPU/RAM: монолит vs. сервиси

ПРОВЕРКА 4 · П4

Убивам pod, прекинувам мрежа

Chaos Mesh + Grafana

MTTD

време до аларм и до лоцирана причина

Обемот: тежината е на едно место

Функционалноста намерно останува иста како кај монолитот — работата е во архитектурата, инфраструктурата и мерењето.

✓ ВЛЕГУВА

- ✓ Kafka настани со transactional outbox и inbox
- ✓ Docker + Kubernetes, локален кластер (k3d)
- ✓ Helm charts + ArgoCD (GitOps)
- ✓ CI/CD со GitHub Actions: тестови, images
- ✓ OpenTelemetry, Prometheus, Grafana, Loki, Tempo
- ✓ Експерименти со k6 и Chaos Mesh

× НЕ ВЛЕГУВА

~~Посебна база по сервис~~

шема по сервис е доволна за изолација

~~Service mesh~~

NetworkPolicy и gateway-от го покриваат основното

~~Хостинг во облак~~

локален кластер е доволен за мерењата

~~Нова функционалност~~

фокусот е на архитектурата

План на работа

нед. 1–3	нед. 4–5	нед. 6–8	нед. 9–11	нед. 12–15
1	2	3	4	5

Стабилизација и CI/CD

- тест gate во CI
- build и push на images
- интеграциски тестови

ИСПОРАКА
зелен pipeline

Helm и GitOps

- Helm charts за сите сервиси
- ArgoCD синхронизација од Git

ИСПОРАКА
deploy од Git

Observability

- OpenTelemetry инструментација
- Loki и Tempo
- dashboards и аларми

ИСПОРАКА
сите три сигнали

Експерименти

- baseline мерења на монолитот
- k6 сценарија
- Chaos Mesh сценарија

ИСПОРАКА
измерени резултати

Анализа и пишување

- обработка на резултатите
- дискусија на компромисите
- текст на трудот

ИСПОРАКА
предаден труд

Што ќе остане од трудот

Референтна архитектура

Production-grade event-driven микросервиси на Kubernetes, со GitOps испорака и целосна observability.

Емпириска споредба

Истата апликација во две архитектури, измерена под оптоварување и при контролирани откази.

Репродуцибилни експерименти

Chaos и load сценарија, dashboards и аларми што секој може повторно да ги изврши.

Документирани компромиси

Eventual consistency, оперативна сложеност и цена во ресурси — кога микросервисите не се исплатат.

ЗАКЛУЧОК

> *Стариот .NET проект ќе го преработам, но тој ми останува и основа за споредба. Ако темата ви изгледа во ред, следуваат доградбата на инфраструктурата, мерењата и пишувањето.*

Ви благодарам. Прашања?

[Име Презиме] · [email]

Kafka · outbox · inbox

Kubernetes · Helm · ArgoCD

метрики · логови · traces

k6 · Chaos Mesh

ДОДАТОК

Можни прашања

Зошто повторно го пишуваш истото?

Функционалноста ја знам, па можам да ги споредувам архитектурите наместо да учам нов домен.

Зошто Java, а не .NET?

Spring Boot и Spring Cloud нудат готов gateway и Kafka интеграција, и тоа е стекот во кој ќе работам понатаму.

Монолит не би бил доволен?

За функционалноста — да, и веќе го имам. Но тогаш нема тема: дистрибуираниот дел е предметот на трудот.

Каде е придонесот?

Споредба на истата апликација во две архитектури, со измерено однесување при откази и оптоварување.

Не е ли обемот преголем?

Затоа јасно кажувам што не влегува. Нема нова функционалност — само архитектура, инфраструктура и мерења.

Зошто Kafka, а не REST повици?

Со директен повик, ако другиот сервис е паднат, паѓа и моето барање. Со настани, барањето поминува.